NUTRIETIQ

Planning de Integración

USDA FoodData Central API

GET /food/{fdcId} — Endpoint de Detalle Nutricional

PARA	PROYECTO	FECHA
Antigravity Dev Team	**Nutrietiq App**	**18 de marzo de 2026**

Stack tecnológico del proyecto

GitHub (control de versiones) | Supabase (base de datos / auth) | Vercel (deploy frontend)

1. Contexto y Objetivo

Este documento define el planning técnico para que el equipo Antigravity integre el endpoint GET /food/{fdcId} de la API pública USDA FoodData Central (FDC) en la aplicación Nutrietiq.

El endpoint permite obtener el perfil nutricional completo de un alimento a partir de su identificador FDC (fdcId), incluyendo macronutrientes, vitaminas, minerales, porciones y metadatos del producto.

1.1 URL del Endpoint

GET	https://api.nal.usda.gov/fdc/v1/food/{fdcId}?api_key=API_KEY

Spec	https://app.swaggerhub.com/apis/fdcnal/food-data_central_api/1.0.1#/FDC/getFood

1.2 Licencia y Costo

- Gratuita: no tiene costo de uso.
- Licencia CC0 1.0 Universal — dominio público (sin restricciones de uso comercial).
- Requiere API Key gratuita obtenida en: fdc.nal.usda.gov/api-key-signup
- Rate limit: 1,000 solicitudes por hora por IP. Solicitar aumento si se necesita mayor volumen.

2. Estructura de Respuesta del Endpoint

La respuesta es un objeto JSON. Los campos varían levemente según el tipo de alimento (dataType), pero el núcleo es siempre el mismo.

2.1 Campos raíz principales

Campo JSON	Tipo	Descripción / Uso en Nutrietiq
`fdcId`	integer	ID único del alimento — clave para llamadas posteriores
`description`	string	Nombre del alimento — mostrar en cabecera de ficha
`dataType`	string	Tipo: Foundation | Branded | SR Legacy | Survey (FNDDS)
`foodCategory`	object	Categoría (Fruits, Dairy, Vegetables…) — útil para filtros
`publicationDate`	string	Fecha de publicación del dato
`foodNutrients`	array	Lista de nutrientes con valores — campo principal
`foodPortions`	array	Porciones definidas (1 cup = 240ml, 1 slice, etc.)
`gtinUpc`	string	Código de barras UPC — solo en Branded Foods — para escaneo QR
`brandOwner`	string	Marca propietaria del producto (solo Branded Foods)
`ingredients`	string	Lista de ingredientes del producto — detección de alérgenos
`servingSize`	number	Tamaño de porción en gramos (solo Branded Foods)

Campo JSON	Tipo	Descripción / Uso en Nutrietiq
`servingSizeUnit`	string	Unidad: g, ml, oz…

** Campos marcados con fondo naranja son prioritarios para la versión inicial de Nutrietiq.*

2.2 Estructura de foodNutrients[]

Cada elemento del array representa un nutriente con su valor por 100g de alimento:

```
// Estructura de un elemento en foodNutrients[]
 {
   "nutrient": {
     "id": 1008,            // ID numérico del nutriente
     "name": "Energy",      // Nombre en inglés
     "unitName": "kcal"     // Unidad de medida
   },
   "amount": 52             // Valor POR 100g del alimento
 }
```

2.3 Nutrientes clave — IDs a mapear en Nutrietiq

ID FDC	Nutriente	Unidad	Uso en Nutrietiq
1008	Energía (Energy)	kcal	Contador calórico principal — meta diaria
1003	Proteína (Protein)	g	Gráfico de macros — meta proteica
1004	Grasa total (Total lipid)	g	Gráfico de macros — control de lípidos
1005	Carbohidratos (Carbohydrate)	g	Gráfico de macros — energía rápida
1079	Fibra dietética (Fiber)	g	Meta diaria de fibra
2000	Azúcares totales (Sugars)	g	Alertas de consumo de azúcar
1093	Sodio (Sodium)	mg	Control de ingesta de sodio
1253	Colesterol	mg	Perfil lipídico completo
1258	Grasa saturada	g	Alertas cardiovasculares
1087	Calcio (Calcium)	mg	Micronutriente — salud ósea
1089	Hierro (Iron)	mg	Micronutriente — anemia
1092	Potasio (Potassium)	mg	Micronutriente — función muscular
1162	Vitamina C	mg	Vitaminas — sistema inmune
1114	Vitamina D	µg	Vitaminas — huesos y sistema inmune
1178	Vitamina B12	µg	Vitaminas — sistema nervioso
1106	Vitamina A (RAE)	µg	Vitaminas — visión e inmunidad

3. Fórmulas de Cálculo

3.1 Conversión por porción

Todos los valores de la API vienen por 100g. Para mostrar valores por porción real en Nutrietiq:

```
// Fórmula base
 const valorPorPorcion = (nutriente.amount * gramos) / 100;

 // Ejemplo: Calorías en 1 taza de leche (244g)
 // amount = 61 kcal/100g
 const calorias = (61 * 244) / 100; // = 148.8 kcal

 // Usar servingSize del objeto para Branded Foods
 const gramos = food.servingSize ?? 100;
 const kcal = (energy.amount * gramos) / 100;
```

3.2 Parámetro ?format para optimizar peticiones

Se recomienda diferenciar el formato según el contexto de uso en Nutrietiq:

Listas	GET /food/{fdcId}?format=abridged → Solo macros + energía. Ideal para resultados de búsqueda y logging rápido.
Detalle	GET /food/{fdcId} → Respuesta completa con todos los micronutrientes. Para ficha nutricional completa del alimento.

3.3 Filtrado de nutrientes específicos

El parámetro ?nutrients= permite filtrar el array devuelto pasando los IDs deseados (máx. 25):

```
// Traer solo macros para contador de ingesta diaria
 GET /food/747447?api_key=KEY&nutrients=1008,1003,1004,1005,1079,2000
```

4. Planning de Integración — Por Fases

El planning se divide en 3 fases progresivas. Se recomienda completar cada fase antes de avanzar a la siguiente.

FASE 1: SETUP Y CONFIGURACIÓN BASE
GitHub → Vercel → Supabase — Estimado: 3–4 días

#	Tarea	Descripción	Responsable	Prioridad	Estimado
1	Obtener API Key FDC	Registro en fdc.nal.usda.gov/api-key-signup. Guardar como DEMO_KEY para dev.	Backend	Alta	0.5d
2	Variable de entorno	Añadir FDC_API_KEY en Vercel (Settings → Environment Variables) y en .env.local del repo.	DevOps	Alta	0.5d
3	Crear servicio API wrapper	Archivo /lib/fdc.ts (o .js) con función getFoodById(fdcId, opts). Centraliza headers, API key y manejo de errores.	Backend	Alta	1d
4	Tabla Supabase: fdc_cache	Crear tabla para cachear respuestas FDC y reducir llamadas (ver schema en sección 5).	Backend	Media	1d
5	Test del endpoint en dev	Verificar respuesta con DEMO_KEY usando fdcId de prueba (ej: 747447 — Apple raw).	Backend	Alta	0.5d

FASE 2: IMPLEMENTACIÓN DEL SERVICIO DE DATOS
Lógica de negocio y transformación — Estimado: 4–5 días

#	Tarea	Descripción	Responsable	Prioridad	Estimado
1	Función transformNutrients()	Convertir array foodNutrients[] a objeto indexado por ID: { 1008: { name, amount, unit }, … }. Facilita el acceso directo en UI.	Backend	Alta	1d
2	Función calcByPortion()	Recibe nutrientes normalizados + gramos de porción. Retorna valores escalados: (amount * gramos) / 100.	Backend	Alta	0.5d
3	Manejo de dataType	Detectar si es BrandedFoodItem para extraer gtinUpc, brandOwner, servingSize. Manejar campos opcionales con defaults.	Backend	Alta	1d
4	Cache en Supabase	Antes de llamar a FDC, buscar en tabla fdc_cache por fdcId. Si existe y tiene menos de 7 días, devolver datos cacheados.	Backend	Media	1d
5	Manejo de errores	Implementar fallback si FDC falla (rate limit 429, timeout). Mostrar	Backend	Alta	1d

#	Tarea	Descripción	Responsable	Prioridad	Estimado
		mensaje amigable en UI. Loggear en Supabase.			
6	API Route en Vercel	Crear /api/food/[fdcId].ts — endpoint interno que consume FDC y devuelve datos transformados al frontend.	Backend	Alta	0.5d

FASE 3: INTEGRACIÓN EN LA UI DE NUTRIETIQ

Componentes y flujo de usuario — Estimado: 5–6 días

#	Tarea	Descripción	Responsable	Prioridad	Estimado
1	Ficha nutricional completa	Componente FoodDetail.tsx. Muestra descripción, categoría, macros, micros y porciones usando datos del endpoint.	Frontend	Alta	2d
2	Selector de porciones	Dropdown con opciones de foodPortions[] (1 cup, 100g, 1 slice). Recalcula todos los valores con calcByPortion().	Frontend	Alta	1d
3	Gráfico de macros	Donut chart con Proteína / Grasa / Carbohidratos (IDs 1003, 1004, 1005) para visualización rápida.	Frontend	Media	1d
4	Badges de marca (Branded)	Si dataType === 'Branded', mostrar brandOwner, código de barras gtinUpc y lista de ingredientes.	Frontend	Media	0.5d
5	Logging de alimentos	Al hacer tap en 'Agregar a mi dieta', guardar fdcId + gramos + fecha en tabla Supabase user_food_log.	Full Stack	Alta	1d
6	Tests de integración	Verificar con al menos 5 fdcIds distintos (Foundation, Branded, SR Legacy). Cubrir casos: sin porciones, sin micronutrientes, Branded sin GTIN.	QA	Alta	0.5d

5. Schema Supabase Recomendado

5.1 Tabla: fdc_cache

Almacena las respuestas de FDC para evitar llamadas repetidas al mismo alimento y reducir el consumo del rate limit.

```
CREATE TABLE fdc_cache (
   id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,
   fdc_id      INTEGER UNIQUE NOT NULL,
   description TEXT NOT NULL,
   data_type   TEXT,
   payload     JSONB NOT NULL,        -- Respuesta completa de FDC
   fetched_at  TIMESTAMPTZ DEFAULT now(),
   expires_at  TIMESTAMPTZ DEFAULT now() + INTERVAL '7 days'
 );

 CREATE INDEX idx_fdc_cache_fdc_id ON fdc_cache(fdc_id);
 CREATE INDEX idx_fdc_cache_expires ON fdc_cache(expires_at);
```

5.2 Tabla: user_food_log

Registra cada alimento consumido por el usuario con su porción y fecha.

```
CREATE TABLE user_food_log (
   id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,
   user_id     UUID REFERENCES auth.users(id) ON DELETE CASCADE,
   fdc_id      INTEGER NOT NULL,
   description TEXT NOT NULL,           -- desnormalizado para rapidez
   grams       NUMERIC(8,2) NOT NULL,   -- gramos consumidos
   meal_type   TEXT CHECK (meal_type IN ('breakfast','lunch','dinner','snack')),
   logged_at   TIMESTAMPTZ DEFAULT now(),
   kcal        NUMERIC(8,2),             -- calculado al guardar
   protein_g   NUMERIC(8,2),
   carbs_g     NUMERIC(8,2),
   fat_g       NUMERIC(8,2)
 );

 ALTER TABLE user_food_log ENABLE ROW LEVEL SECURITY;
 CREATE POLICY "Users see own logs" ON user_food_log
   FOR ALL USING (auth.uid() = user_id);
```

6. Código de Referencia

6.1 Wrapper del servicio FDC (/lib/fdc.ts)

```
const FDC_BASE = "https://api.nal.usda.gov/fdc/v1";
 const API_KEY  = process.env.FDC_API_KEY ?? "DEMO_KEY";

 export async function getFoodById(
   fdcId: number,
   opts: { format?: "abridged" | "full"; nutrients?: number[] } = {}
 ) {
   const params = new URLSearchParams({ api_key: API_KEY });
   if (opts.format === "abridged") params.set("format", "abridged");
   if (opts.nutrients?.length) params.set("nutrients", opts.nutrients.join(","));

   const res = await fetch(`${FDC_BASE}/food/${fdcId}?${params}`, {
     next: { revalidate: 86400 }  // Next.js ISR: 24h cache en edge
   });

   if (!res.ok) throw new Error(`FDC error: ${res.status}`);
   return res.json();
 }
```

6.2 Función de transformación de nutrientes

```
const MACRO_IDS = { energy:1008, protein:1003, fat:1004, carbs:1005,
                    fiber:1079, sugars:2000, sodium:1093 };

 export function transformNutrients(foodNutrients: any[]) {
   return foodNutrients.reduce((acc, fn) => {
     acc[fn.nutrient.id] = {
       name:   fn.nutrient.name,
       amount: fn.amount,
       unit:   fn.nutrient.unitName
     };
     return acc;
   }, {} as Record<number, { name: string; amount: number; unit: string }>);
 }

 export function calcByPortion(nutrients: ReturnType<typeof transformNutrients>,
                               grams: number) {
   const scale = grams / 100;
   return Object.fromEntries(
     Object.entries(nutrients).map(([id, n]) => [
       id, { ...n, amount: Math.round(n.amount * scale * 100) / 100 }
     ])
   );
 }
```

6.3 API Route en Next.js (/app/api/food/[fdcId]/route.ts)

```
import { getFoodById, transformNutrients } from "@/lib/fdc";
 import { createClient } from "@/lib/supabase/server";

 export async function GET(req: Request,
   { params }: { params: { fdcId: string } }
 ) {
   const fdcId = parseInt(params.fdcId);
   const supabase = createClient();
```

```
  // 1. Buscar en cache de Supabase
  const { data: cached } = await supabase
    .from("fdc_cache")
    .select("payload")
    .eq("fdc_id", fdcId)
    .gt("expires_at", new Date().toISOString())
    .single();

  if (cached) return Response.json(cached.payload);

  // 2. Llamar a FDC API
  const food = await getFoodById(fdcId);
  const nutrients = transformNutrients(food.foodNutrients ?? []);

  // 3. Guardar en cache
  await supabase.from("fdc_cache").upsert({
    fdc_id: fdcId,
    description: food.description,
    data_type: food.dataType,
    payload: { ...food, _nutrients: nutrients }
  });

  return Response.json({ ...food, _nutrients: nutrients });
}
```

7. Resumen de Estimaciones

Fase	Descripción	Duración	Prioridad
Fase 1	Setup y configuración base (API Key, env vars, tabla Supabase, wrapper)	3–4 días	Alta
Fase 2	Implementación del servicio de datos (transform, cache, manejo de errores)	4–5 días	Alta
Fase 3	Integración en UI (ficha, selector porciones, gráfico macros, logging)	5–6 días	Alta
TOTAL	Integración completa del endpoint en Nutrietiq	**12–15 días**	—

7.1 Dependencias y Consideraciones

- **Bloqueante:** La API Key de FDC debe estar configurada en Vercel antes de comenzar la Fase 1.
- **Bloqueante:** El schema de Supabase (Fase 1) es prerrequisito para la Fase 2 (cache) y Fase 3 (logging).
- **Infra:** Coordinar con Vercel las variables de entorno para ambientes staging y production.
- **Escalabilidad:** Si el volumen de usuarios supera 800 búsquedas/hora, solicitar rate limit aumentado a FDC.
- **Seguridad:** Guardar DEMO_KEY solo en .env.local — nunca commitear al repositorio GitHub.
- **Legal:** Citar la fuente USDA FoodData Central en la sección 'Acerca de' de la app (requerimiento de licencia CC0).

8. Recursos y Referencias

Recurso	URL / Referencia
Spec OpenAPI (Swagger)	`https://app.swaggerhub.com/apis/fdcnal/food-data_central_api/1.0.1`
Guía oficial API FDC	`https://fdc.nal.usda.gov/api-guide`
Registro API Key	`https://fdc.nal.usda.gov/api-key-signup`
Documentación datos	`https://fdc.nal.usda.gov/data-documentation`
Rate limits data.gov	`https://api.data.gov/docs/rate-limits`
Supabase Docs	`https://supabase.com/docs`
Vercel Env Variables	`https://vercel.com/docs/concepts/projects/environment-variables`

Documento preparado para

ANTIGRAVITY

Integración FoodData Central API — Nutrietiq App